# DVFS, Governors & the Energy-Performance Pareto

## 1 Introduction

Dynamic Voltage and Frequency Scaling (DVFS) and CPU governors are key mechanisms that influence the trade-off between performance, power, and energy efficiency in modern systems. This lab extends the concepts introduced in Lab 1 by integrating frequency control, power measurements, and Pareto analysis.
The lab aims to:

- Explore the impact of CPU frequency scaling and governor selection on performance, power and energy.
- Compare external and internal power measurements.
- Build and read Pareto curves (energy vs. time and power vs. time) from your own measurements.
- Understand why compute-heavy and memory-heavy code react differently to frequency.
- Justify a design choice with data and translate it into battery life.

## 2 Design the "Balanced" Mode

Your group works for a company that sells a small battery-powered device built on a Raspberry Pi 5. Like a phone, the device has three power modes:

- **Performance** – uses the `performance` governor (always 2400 MHz).
- **Battery saver** – uses the `powersave` governor (always 1500 MHz).
- **Balanced** – *does not exist yet. Designing it is your job.*

Two apps run on the device: **MatMul** plays the role of a *compute-heavy* app and **Triad** the role of a *memory-heavy* app. Balanced must make sense for both.

Your group will:

1. Measure the two existing modes and the built-in `ondemand` and `schedutil` governors (more details about these governors ...).
2. Measure a fixed set of frequencies (the same for every group) and build the **Pareto curves** (energy vs. time and power vs. time) for both apps.
3. **Choose Balanced on the Pareto front** and explain the trade-off with numbers: how much time do you give up, and how much energy or power do you save, compared with Performance? There is no fixed deadline in this lab (deadlines come in Lab 4): you decide what "balanced" means, and you must justify it with your curves.
4. Translate your three modes into what a user cares about: the impact on the **battery**.

## 3 Lab Preparation

1. **Pre-lab task:** Complete the **Pre-Lab 2 assignment** in Canvas before attending the session.
2. **Review of CPU Governors:** Before coming to the lab, review the main CPU governors available on the Raspberry Pi (`performance`, `powersave`, `ondemand`, `schedutil`, `conservative`, `userspace`) and understand their basic behavior.



UNIVERSITY OF GOTHENBURG



CHALMERS UNIVERSITY OF TECHNOLOGY

3. **Setup the measurement environment:**
   - Connect the Raspberry Pi power cable to the external power meter and plug it into the wall socket.
   - Verify that the Raspberry Pi boots correctly and that the power meter display is active.
4. **Copying the commands:** All commands of this document are also in the text file below. **Copy the commands from this file, not from the PDF**: copying from a PDF can change characters (e.g. quotes, dashes, underscores) and the command will fail.

   ```
   /home/dat278/labs/lab2/commands.txt
   ```
5. **Software and permissions:** The image already contains everything you need: `python3` with `matplotlib` and `numpy`, `gcc`, and `vcgencmd`. You can check it with:

   ```
   python3 -c "import matplotlib, numpy; print('python libraries OK')"
   gcc --version
   vcgencmd measure_temp
   ```

   Changing the CPU governor or frequency requires administrator rights, so `set_governor.sh` and `set_freq.sh` **must be run with** `sudo`. All other scripts run without `sudo`.
6. **Directory layout:** All scripts, data, kernels, and logs for this lab are located under:

   ```
   /home/dat278/labs/lab2
   ```

   The main directory contains the following elements:

   - **Scripts:**
     - `status.sh` – shows the current governor, frequency, temperature and throttling state. Use it whenever you are unsure what the Pi is set to.
     - `set_governor.sh <governor>` – selects a governor (needs `sudo`).
     - `set_freq.sh <MHz>` – locks the CPU to one frequency using the `userspace` governor (needs `sudo`).
     - `run.sh matmul|triad` – runs one kernel *once* at the current setting, measures time, average power and energy, and adds one row to `data/results.csv`.
     - `check_results.py` – finds and removes disturbed runs, and tells you which runs to repeat.
     - `plot_pareto.py <kernel>` – prints a summary table and saves the Pareto plots in `plots/`.
     - `commands.txt` – all the commands of this document.
   - **lab1/** – the Lab 1 roofline scripts and kernels, used in Task 1.
   - **utils/** – `power_wrapper.sh` (measures time, average power and energy of a command) and `power_sampler.sh` (reads the power of the Pi's power-management chip, PMIC).
   - **kernels/** – the two apps and `build_kernels.sh`:
     - `matmul/matmul_exec` – compute-bound kernel (compute-heavy app).
     - `triad/triad` – memory-bound kernel (memory-heavy app).
   - **data/results.csv** – created by `run.sh`: one row per run. Runs removed by `check_results.py` are kept in **data/removed.csv**.
   - **plots/** – created by `plot_pareto.py`.

# 4 In-Lab Tasks

## Task 1 – Warm-up: Governors, Frequencies and the Roofline

This task checks that everything works (scripts, `sudo`, Python libraries) and recalls Lab 1.

1. Go to the lab folder and look at the current state of the CPU:

```
cd /home/dat278/labs/lab2
./status.sh
```

2. Try the governors and watch how the frequency changes:

```
sudo ./set_governor.sh performance
sudo ./set_governor.sh powersave
sudo ./set_governor.sh ondemand
```

3. Run the Roofline experiment from Lab 1 at the minimum available frequency, using the copy of the Lab 1 files in `lab2/lab1`:

```
cd /home/dat278/labs/lab2
sudo ./set_freq.sh 1500
cd lab1/roofline_emp
./run_peak_flops.sh
./run_stream_bench.sh
cd ../comp-b_vs_mem-b
./run_kernels.sh
cd ..
nano inp/hw_specs.json (or update in some way)
nano inp/kernel_specs.json (or update in some way)
python3 draw_roofline.py
```

   As in Lab 1, update the two JSON files with the values you just measured: `hw_specs.json` (peak GFLOP/s and memory bandwidth) and `kernel_specs.json` (`measured_gflops` of each kernel; do not change the operational intensity). The plot is saved in `lab1/out/`.

4. Repeat at the maximum available frequency, updating the two JSON files again:

```
cd /home/dat278/labs/lab2
sudo ./set_freq.sh 2400
cd lab1/roofline_emp
./run_peak_flops.sh
./run_stream_bench.sh
cd ../comp-b_vs_mem-b
./run_kernels.sh
cd ..
nano inp/hw_specs.json (or update in some way
nano inp/kernel_specs.json (or update in some way
python3 draw_roofline.py
```

5. Compare the Roofline plots obtained at both frequencies.

**In the lab report: (100 words)**

1. How does lowering the frequency affect the compute and memory roofs?
2. Which region (memory or compute-bound) is more affected?
3. Discuss what changes in performance you observed and why.



UNIVERSITY OF GOTHENBURG



CHALMERS UNIVERSITY OF TECHNOLOGY

## Task 2 – Comparing External and Internal Power Measurements

1. Ensure the Raspberry Pi is connected through the external power meter.
2. Run a simple idle measurement for 10 seconds. The script measures the power reported by the PMIC (internal measurement):

```
cd /home/dat278/labs/lab2
./utils/power_wrapper.sh sleep 10
```

3. Record both the external and internal power readings and compute their difference.

**In the lab report: (100 words)**

1. Which reading is higher? The internal or the external one?
2. What hardware components are included in the external measurement but not in the internal one?

## Task 3 – Measure the Existing Modes

**Turn off the graphical interface (GUI) for Tasks 3 and 4 – mandatory.** Programs running in the background (VS Code, web browser, the desktop itself) use CPU time and power, and they can completely hide the effect you want to measure. No reboot is needed:

`sudo systemctl isolate multi-user.target`

If the screen shows only a blinking dash, press `Ctrl+Alt+F2` (or `Ctrl+Alt+F3`) to get a text console. Log in as `dat278` and go to the lab folder with `cd /home/dat278/labs/lab2`. Copying is not possible in the console, so type the commands (they are short). Do not use the Pi while `run.sh` is running. You will turn the GUI on again at the end of Task 4.

Each call of `run.sh` runs the kernel once (MatMul ≈ 9–14 s, Triad ≈ 4 s). To make the results comparable between groups, every run starts from the same condition: `run.sh` **does not start while the CPU is above 55 °C**; it waits until the CPU has cooled down. **Every setting is measured twice**, so that disturbed runs can be found and removed in Task 4.

1. Performance mode:

```
sudo ./set_governor.sh performance
./run.sh matmul
./run.sh matmul
./run.sh triad
./run.sh triad
```

2. Battery saver mode:

```
sudo ./set_governor.sh powersave
./run.sh matmul
./run.sh matmul
./run.sh triad
./run.sh triad
```

3. The built-in dynamic governors:

```
sudo ./set_governor.sh ondemand
./run.sh matmul
./run.sh matmul
./run.sh triad
./run.sh triad
sudo ./set_governor.sh schedutil
./run.sh matmul
./run.sh matmul
./run.sh triad
./run.sh triad
```

4. Look at your measurements (one row per run):

```
cat data/results.csv
```

**In the lab report: (100 words)**

1. For each governor and kernel you have two runs: fill in the table in the report with the time, average power and energy of *one* of them. Comment briefly: how close are the two runs?
2. Which governor gives the best time? The lowest power? The lowest energy?
3. Does Battery saver really save battery? Explain your answer using $E = P \cdot t$.
4. Why do `ondemand` and `schedutil` behave so similarly to one of the fixed modes for these programs?

## Task 4 – Build the Pareto Curves

1. You already have the four governors from Task 3 (powersave = 1500 MHz, performance = 2400 MHz). Now measure the three fixed frequencies, again twice per kernel:

```
sudo ./set_freq.sh 1800
./run.sh matmul
./run.sh matmul
./run.sh triad
./run.sh triad
sudo ./set_freq.sh 2000
./run.sh matmul
./run.sh matmul
./run.sh triad
./run.sh triad
sudo ./set_freq.sh 2200
./run.sh matmul
./run.sh matmul
./run.sh triad
./run.sh triad
```

2. **Check your measurements:**

```
python3 check_results.py
```

For every setting, the script compares the two runs. Background activity can only make a run slower and more expensive, so a run that is more than 5% slower or 5% more expensive than the other one is **removed** (it is moved to `data/removed.csv`; throttled runs are removed too). The script then prints the exact commands to measure the removed runs again. Run them, then run `check_results.py` again, until it prints `All measurements OK`.

3. Set the CPU back to the default governor and turn the GUI on again:

```
sudo ./set_governor.sh ondemand
sudo systemctl isolate graphical.target
```

4. **Build the Pareto curves** (open a terminal and go to `/home/dat278/labs/lab2` again):

```
python3 plot_pareto.py matmul
python3 plot_pareto.py triad
```

The plots are saved in `plots/pareto_matmul.png` and `plots/pareto_triad.png`. Each point is the average of the two runs of that setting. Each figure has two plots: energy vs. time (left) and average power vs. time (right). In both plots the best corner is **bottom-left**. Blue points are Pareto-optimal (no other setting is both faster and cheaper); grey points are dominated. Measurements are noisy (about ±3%): points closer than that are practically equal, keep it in mind when you read the front. The table printed in the terminal gives, for every setting, the slowdown and the energy change relative to Performance.

**In the lab report: (100 words)**

1. Include the 2 images inside the report.
2. Which settings are Pareto-optimal in the energy–time plot? And in the power–time plot? Why are the two fronts different?
3. Where do Performance, Battery saver, `ondemand` and `schedutil` lie with respect to the fronts?
4. How do the MatMul and Triad curves differ? Relate your answer to the roofline of Task 1 (compute-bound vs. memory-bound).

## Task 5 – Design the Balanced Mode

This is your design decision: different groups may choose differently, as long as the choice is justified by the curves.

1. Using the plots and the table of Task 4, choose Balanced for each app among the settings on the Pareto front. A good candidate is the *knee* of the curve: the point after which slowing down further saves very little.

**In the lab report: (100 words)**

1. Which frequency did you choose for MatMul and for Triad? Explain the trade-off with numbers from the table (e.g. "X% slower than Performance for Y% less energy").
2. Is there a clear knee for both apps? Why or why not?

## Task 6 – What Does It Mean for the Battery?

Your device has a **10 000 mAh** battery with a voltage of **5 V**. The energy stored in the battery is:

$$E_{\text{battery}} = 10\,\text{Ah} \times 5\,\text{V} = 50\,\text{Wh} = 50 \times 3600\,\text{J} = 180\,000\,\text{J}$$

A user can ask two different questions:

- **How many hours does the battery last while the app is running?** $\text{hours} = \dfrac{E_{\text{battery}}}{P_{\text{avg}} \times 3600}$ (depends on **power**)
- **How many times can I run the app on one charge?** $\text{runs} = \dfrac{E_{\text{battery}}}{E_{\text{run}}}$ (depends on **energy per run**)

1. Using the averaged values printed by `plot_pareto.py`, compute by hand (calculator) the hours and the runs per charge for Performance, Battery saver and your Balanced mode, for both apps.

**In the lab report: (100 words)**

1. Fill in the table in the report with the hours and runs per charge of the three modes for both apps.
2. Which mode would you recommend to a user who wants the battery to *last all day*? And to a user who wants to *process as many jobs as possible* on one charge? Why can the two answers be different?
3. The values are based on the internal (PMIC) power. Would the real battery last longer or shorter? Use your result of Task 2.